Detecting Mutated DNA Motifs: Hamming-Distance Matching Versus k-mer Machine Learning Models

Jungbin Yoon

***Abstract*—Deoxyribonucleic acid (DNA) motifs are short, recurring nucleotide patterns, many of which serve as transcription factor binding sites, yet real binding sites rarely match a motif exactly. This study asked whether machine learning (ML) models detect mutated motifs more accurately than classical pattern-matching algorithms as mutation rates increase. Synthetic datasets of 1,000 random 100-nucleotide sequences were generated in which about half of the sequences carried a 12-nucleotide motif mutated at per-nucleotide rates from 0% to 50%, with 30 independent datasets per rate (180 in total). Exact matching, Hamming-distance matching, logistic regression and random forest classifiers built on k-mer frequency features, and a convolutional neural network were compared on identical held-out test sequences. Exact matching was perfect without mutations but fell to 0.635 accuracy at a 10% mutation rate. Hamming-distance matching was the most accurate method at every nonzero mutation rate (0.969 at 10% and 0.739 at 30%), and no other method significantly exceeded it once mutations were present. Lengthening the k-mer from 3 to 6 nucleotides raised the ML models to near-perfect accuracy without mutations but did not close the gap under mutation, and the convolutional network narrowed that gap without overturning it. A probabilistic model predicted classical accuracy to within 0.009 across mutation rates and to within 0.018 across mismatch thresholds from 0 to 6. When the target motif is known, a detector matched to the mutation process is therefore difficult to beat, and the limiting factor for the ML models was their sequence representation rather than the classifier.**

I. Introduction

Gene expression is controlled in large part by transcription factors (TFs), proteins that recognize short DNA sequences called motifs that are typically only 6 to 20 nucleotides long [1], [2]. Because TFs tolerate some variation, their binding sites are degenerate: functional sites in a genome often differ from the consensus motif at several positions [3], [4]. Locating motif instances despite such variation is therefore a core problem in bioinformatics.

Computational approaches range from exact string matching and consensus- or k-mer-based searches [5] to machine learning (ML) models that learn sequence patterns directly from data, including support vector machines built on gapped k-mer features [6] and deep neural networks [7], [8]. ML methods are often assumed to handle noisy, variable sequences better than fixed rules, but that advantage is not guaranteed, particularly for problems in which the target pattern is already known.

This project grew out of Rosalind, an online platform that teaches bioinformatics through programming problems [9]. Its problems *Finding a Motif in DNA* (SUBS), *Counting Point Mutations* (HAMM), and *k-Mer Composition* (KMER) introduce exact matching, Hamming distance, and k-mer representations, respectively [10]. Working through them raised a question: if mutations make exact matching fail, can ML recognize motifs that have been altered? The research question was whether ML models detect mutated DNA motifs more accurately than classical motif-finding algorithms as mutation rates increase, and the initial hypothesis was that they would at higher mutation rates.

II. Background

The Hamming distance between two sequences of equal length is the number of positions at which they differ [11]. Searching for windows within d mismatches of a motif is central to the planted (l, d)-motif problem, in which a motif of length l is implanted with up to d substitutions into random background sequences [12]. Motif-discovery tools such as MEME [13] and position weight matrix models [4] address the harder task of describing motifs that are not known in advance; this study treats the motif as known, isolating the detection step. ML approaches instead convert sequences into numerical features. The simplest is a k-mer frequency vector that counts every substring of length k; it is order-free, recording how often each k-mer occurs but not where [6]. Convolutional neural networks (CNNs) avoid this loss by scanning learned filters along the sequence, much like a flexible motif search [7], [14].

III. Methods

A. Pilot Study and Redesign

A pilot experiment used a single dataset of 1,000 random 50-nucleotide (nt) sequences whose positive sequences carried the 6-nt motif ATGCTA mutated at a 30% rate. All methods scored close to chance (0.518 to 0.633). The weak Hamming results have a simple cause: a random 6-mer lies within three mismatches of a given 6-nt motif with probability 694/4,096 ≈ 0.17, so nearly every 50-nt random sequence, with its 45 windows, contains a false match. The motif was therefore lengthened to 12 nt (ATGCTAGCGTAC), sequences to 100 nt, and 30 independent datasets were generated per mutation rate so that real differences could be separated from random variation.

B. Dataset Generation

All datasets were generated in Python with a fixed random seed. For each of the 1,000 sequences in a dataset, a class label was drawn at random with equal probability (label 1, motif present; label 0, motif absent). Every sequence began as 100 nt drawn uniformly from A, C, G, and T. For a positive sequence, each of the 12 motif positions was independently substituted, with probability p (the per-nucleotide mutation rate), by one of the three other nucleotides; the mutated motif then overwrote a randomly positioned 12-nt window. Negative sequences received no insertion. Table I lists the parameters and Fig. 1 summarizes the workflow.

TABLE I. Dataset Parameters

Parameter	Value
Sequence length	100 nt
Motif (length)	ATGCTAGCGTAC (12 nt)
Sequences per dataset	1,000

Parameter	Value
Mutation rates, p	0, 10, 20, 30, 40, 50%
Datasets per rate	30
Total datasets	180

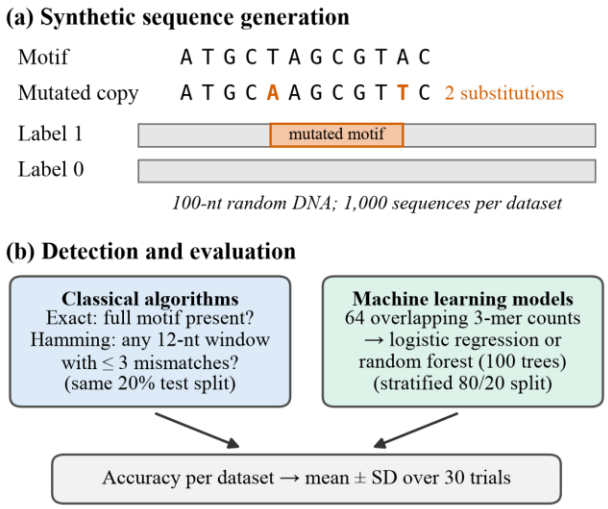


Figure 1. Study design. (a) Positive sequences carry a copy of the 12-nt motif with random substitutions (orange) at a random position; negative sequences are random DNA. (b) All methods were scored on the same held-out test sequences.

C. Classical Detection Algorithms

Exact matching (Rosalind SUBS) labeled a sequence positive if it contained the unmutated motif. Hamming-distance matching (Rosalind HAMM) slid a 12-nt window across all 89 positions of a sequence and labeled the sequence positive if any window w satisfied

$$d_{\mathrm{H}}(w, m) = \sum_{i=1}^{12} [w_i \neq m_i] \leq 3, \qquad (1)$$

where m is the motif, the sum runs over the 12 positions i, and $[\cdot]$ equals 1 when its condition is true and 0 otherwise. A threshold of three mismatches (25% of the motif) was fixed in advance for the main comparison, and thresholds $d = 0$ to 6 were swept separately to test whether that choice was optimal.

D. k-mer Machine Learning Models

Each sequence was converted into a feature vector containing the counts of all 4^k possible overlapping k-mers (Rosalind KMER). Two classifiers were trained with scikit-learn [15]: logistic regression (default L2 regularization, at most 500 iterations) and a random forest of 100 decision trees combined by majority vote [16]. The main comparison used $k = 3$, giving 64 features, and k was then swept from 2 to 6.

E. Convolutional Neural Network

Because k-mer counts discard position, a position-aware model was added as a control. A one-dimensional CNN was implemented in PyTorch with a single convolutional layer of 16 filters of width 12 applied to the one-hot encoded sequence, a rectified linear activation, global max pooling, and one dense output unit. It was trained for 30 epochs with the Adam optimizer, a learning rate of 0.001, and a batch size of 64. Global max pooling records how well each filter matches anywhere in the sequence, which is structurally similar to a sliding-window search.

F. Evaluation and Statistical Analysis

Each dataset was divided into 800 training and 200 test sequences using a stratified split with a fixed seed. Every method, including the untrained classical algorithms, was scored on the same 200 test sequences, so all accuracies are directly comparable. Accuracy (the fraction of correctly classified sequences, with chance at about 0.50), recall, and false positive rate were recorded for each dataset, and means and standard deviations (SD) were computed across the 30 datasets per mutation rate. Methods were compared using two-sided Wilcoxon signed-rank tests paired by dataset (n = 30).

G. Theoretical Accuracy of the Classical Methods

Because the data-generating process is fully specified, the expected accuracy of a matcher with threshold d can be derived. The probability that a single random 12-nt window lies within d mismatches of the motif is

$$q_d = \frac{\sum_{k=0}^{d} \binom{12}{k} 3^k}{4^{12}}, \qquad (2)$$

giving $q_0 = 6.0 \times 10^{-8}$ and $q_3 = 3.9 \times 10^{-4}$. Treating the 89 windows as independent, a negative sequence is falsely flagged with probability $F_d = 1 - (1 - q_d)^{89}$, which is 3.4% for $d = 3$. The number of substitutions B in an inserted motif follows a binomial distribution with 12 trials and probability p, so a positive sequence is detected with probability $T_d \approx \mathrm{P}(B \leq d) + [1 - \mathrm{P}(B \leq d)]\, F_d$. For balanced classes, the expected accuracy is

$$A_d(p) \approx \tfrac{1}{2}(1 - F_d) + \tfrac{1}{2} T_d. \qquad (3)$$

IV. Results

A. Accuracy Across Mutation Rates

All methods lost accuracy as the mutation rate increased, but at very different speeds (Fig. 2, Table II). Exact matching was perfect at $p = 0\%$ but dropped to 0.635 at 10% and 0.542 at 20%, and was indistinguishable from chance at 30% and above. Hamming matching scored 0.982 at 0% and was the most accurate method at every nonzero rate: 0.969 at 10%, 0.890 at 20%, and 0.739 at 30%, before approaching chance at 50% (0.534). With 3-mer features, the ML models trailed it in all 30 paired datasets at every rate from 0% to 40% (Wilcoxon $p \leq 1.7 \times 10^{-6}$) and in at least 21 of 30 datasets at 50% ($p \leq 2.6 \times 10^{-4}$); all these differences survive Bonferroni correction. Random forest beat logistic regression without mutations (0.934 vs. 0.835, $p = 1.7 \times 10^{-6}$), but the two were close at every other rate.

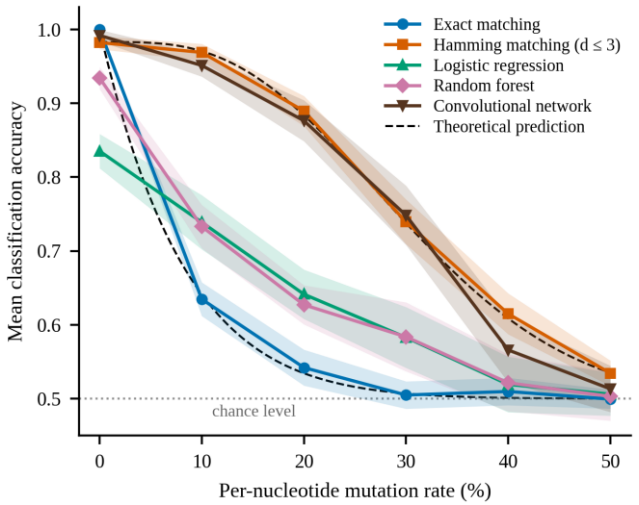


Figure 2. Mean accuracy across 30 datasets per mutation rate. Shaded bands show ±1 SD, dashed curves show the theoretical accuracy of exact and Hamming matching from (3), and the dotted line marks chance (0.50). The machine learning models use 3-mer features.

TABLE II. MEAN ACCURACY (± SD) ON IDENTICAL TEST SETS

Rate (%)	Exact	Hamming	Log. reg.	Forest	CNN
0	**1.000 ±0.000**	0.982 ±0.009	0.835 ±0.023	0.934 ±0.015	0.992 ±0.008
10	0.635 ±0.023	**0.969 ±0.012**	0.739 ±0.037	0.733 ±0.029	0.951 ±0.015
20	0.542 ±0.024	**0.890 ±0.020**	0.641 ±0.033	0.627 ±0.026	0.876 ±0.028
30	0.505 ±0.019	0.739 ±0.030	0.583 ±0.041	0.584 ±0.047	**0.748 ±0.040**
40	0.510 ±0.019	**0.615 ±0.028**	0.519 ±0.038	0.521 ±0.039	0.566 ±0.044
50	0.499 ±0.013	**0.534 ±0.017**	0.506 ±0.029	0.503 ±0.033	0.513 ±0.031

a. Highest mean in each row is in bold; the CNN advantage at 30% is not significant (p = 0.15). Logistic regression and random forest use 3-mer features.

B. Two Kinds of Error

Accuracy alone hides how the methods fail (Fig. 3). Both classical algorithms were highly specific: their false positive rate stayed at 0.000 for exact matching and near 0.030 for Hamming matching at every mutation rate, matching the 3.4% predicted by the model above. Their accuracy loss came entirely from missed motifs, with Hamming recall falling from 1.000 at 0% to 0.508 at 30% and 0.101 at 50%. The ML models behaved in the opposite way: their false positive rate climbed from 0.168 at 0% to 0.498 at 50% for logistic regression, so at high mutation rates they were not detecting weak motifs but guessing. In an application where flagged sites are followed up experimentally, this difference matters more than the accuracy gap itself.

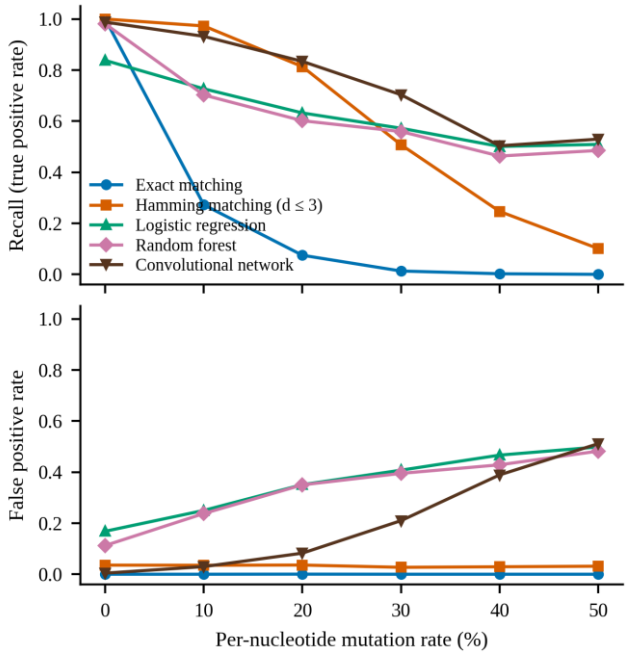


Figure 3. Recall and false positive rate for each method. The classical algorithms miss mutated motifs but rarely flag random sequences, whereas the machine learning models trade sharply rising false positives for recall.

C. Agreement With Theory and the Mismatch Threshold

The model in (3) reproduced the classical results closely (Fig. 2, dashed curves), differing from the observed means by at most 0.009 across mutation rates. Extending the comparison to all 42 combinations of threshold $d = 0$ to 6 and mutation rate, the maximum deviation was 0.018, so the same two-parameter description holds well away from the conditions it was checked on. The sweep (Fig. 4) also shows that no single threshold is best everywhere: $d = 0$ or 1 is optimal without mutations, $d = 3$ at 10% and 20% (0.969 and 0.890), and $d = 4$ from 30% onward (0.775 at 30% and 0.670 at 40%). The threshold used in the main comparison was therefore a good but not optimal choice at the highest mutation rates, and the theory predicts the optimum without any additional experiments.

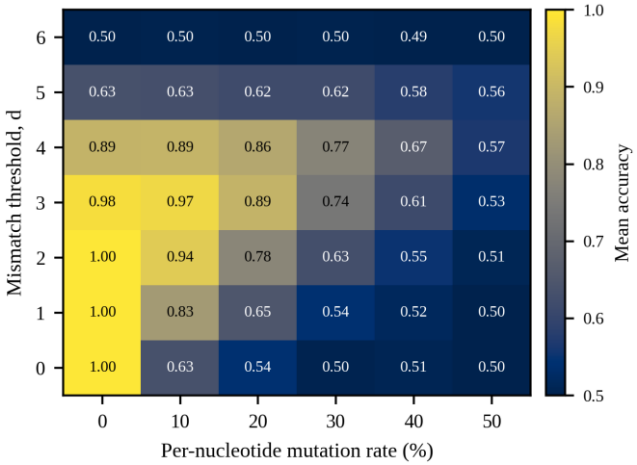


Figure 4. Mean accuracy of Hamming matching for mismatch thresholds $d = 0$ to 6. The best threshold increases with the mutation rate, from 0 without mutations to 4 at 30% and above.

D. Sequence Representation Versus Classifier

Lengthening the k-mer improved the ML models substantially (Fig. 5). At p = 0%, accuracy rose from 0.835 (logistic regression) and 0.934 (random forest) with 3-mers to 1.000 with 6-mers, exceeding Hamming matching in 28 of 30 datasets ($p = 3.4 \times 10^{-6}$). This is expected: a 6-mer is long enough to be rare by chance, so an unmutated motif leaves an almost unique signature. Under mutation, however, the gain shrank quickly; the best k at each rate still trailed Hamming matching in every one of 30 datasets at 10% to 40% (for example 0.884 vs. 0.969 at 10% and 0.752 vs. 0.890 at 20%). The CNN, which preserves position, came closest: it exceeded Hamming matching at 0% (0.992 vs. 0.982, $p = 3.0 \times 10^{-4}$), matched it at 30% (0.748 vs. 0.739, $p = 0.15$), but remained below it at 10%, 20%, 40%, and 50%.

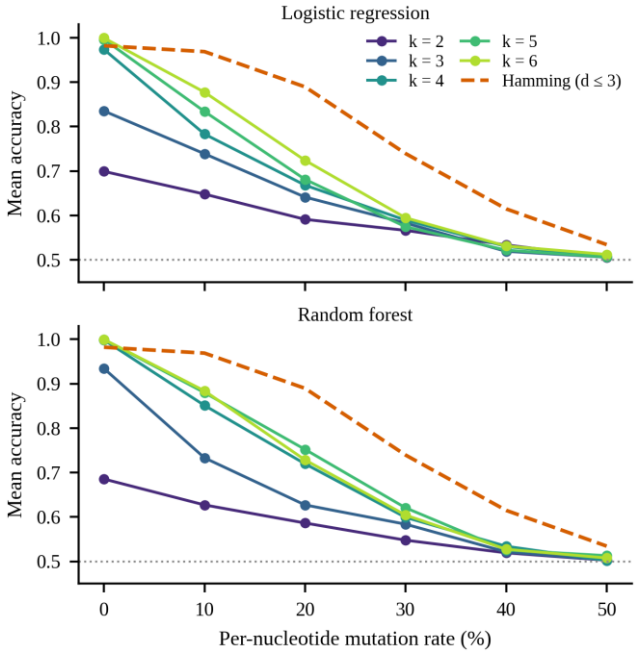


Figure 5. Effect of k-mer length on both machine learning models, with Hamming matching (dashed) for reference. Longer k-mers help most when the motif is unmutated.

V. Discussion

The hypothesis that ML models would outperform classical algorithms at high mutation rates was not supported. Hamming matching was the most accurate method at every nonzero mutation rate, and across k-mer lengths from 2 to 6, two classifiers, and a position-aware network, nothing significantly exceeded it once mutations were present.

The k-mer sweep and the CNN show why, and they point at the representation rather than the classifier. A 12-nt motif contributes only 10 of the 98 3-mers in a sequence, so its signal is a small rise in the counts of ten specific 3-mers above a random background of about 1.5 counts each. Each substitution destroys up to three of those 3-mers, and a given motif 3-mer survives with probability $(1 - p)^3$, so at p = 30% only about 3.4 of the 10 remain. Longer k-mers carry a sharper signature but are even more fragile: a 6-mer survives with probability $(1 - p)^6$, which is why $k = 6$ was near-perfect at 0% and no better than $k = 3$ by 30%. The CNN, whose filters slide along the sequence, recovered much of the gap, confirming that position information was the missing ingredient. Hamming matching simply encodes that information exactly, because its comparison mirrors how the data were generated.

Exact matching shows the opposite trade-off: no false positives, no tolerance for variation. Together with the threshold sweep, this illustrates a general point about detection. Any of these methods can be tuned along a recall-versus-false-positive curve, and the method that wins is the one whose assumptions match the process that generated the variation. ML is likely to be most valuable where that process is unknown, for example when the motif itself must be discovered, when several motifs interact, or when binding depends on the surrounding sequence [3], [7].

VI. Limitations and Future Work

All sequences were synthetic, with uniform base composition and independent substitutions, whereas real genomes contain compositional bias, insertions and deletions, and repetitive elements. Only one motif of one length was tested, and insertions or deletions would break the fixed-width window that Hamming matching depends on, likely favoring more flexible models. The ML hyperparameters were not tuned, and the CNN used a single small architecture trained for a fixed number of epochs, so it represents a lower bound on what a position-aware model can do. Future work should test experimentally identified TF binding sites, where the motif is less precisely defined, and compare methods on sequences containing indels as well as substitutions.

VII. Conclusion

This study compared classical algorithms, k-mer-based ML classifiers, and a convolutional network for detecting a known 12-nt DNA motif at mutation rates from 0% to 50%. Exact matching was perfect without mutations but failed rapidly once they were introduced. Hamming-distance matching was the most accurate method at every nonzero mutation rate, no other method significantly exceeded it, and a simple probabilistic model predicted its accuracy to within 0.018 across every threshold and mutation rate tested. Longer k-mers and a position-aware network narrowed the gap, showing that the ML models were limited by how they represented sequences rather than by the classifiers themselves. When the target motif is known, matching the detector to the mutation process matters more than model complexity.

Acknowledgment

The author thanks the creators of the Rosalind platform, whose problems inspired this project.

References

[1] S. A. Lambert *et al.*, "The human transcription factors," *Cell*, vol. 172, no. 4, pp. 650–665, 2018.

[2] W. W. Wasserman and A. Sandelin, "Applied bioinformatics for the identification of regulatory elements," *Nat. Rev. Genet.*, vol. 5, no. 4, pp. 276–287, 2004.

[3] S. Inukai, K. H. Kock, and M. L. Bulyk, "Transcription factor–DNA binding: Beyond binding site motifs," *Curr. Opin. Genet. Dev.*, vol. 43, pp. 110–119, 2017, doi: 10.1016/j.gde.2017.02.007.

[4] G. D. Stormo, "DNA binding sites: Representation and discovery," *Bioinformatics*, vol. 16, no. 1, pp. 16–23, 2000.

[5] V. Boeva, “Analysis of genomic sequence motifs for deciphering transcription factor binding and transcriptional regulation in eukaryotic cells,” *Front. Genet.*, vol. 7, Art. no. 24, 2016, doi: 10.3389/fgene.2016.00024.
[6] M. Ghandi, D. Lee, M. Mohammad-Noori, and M. A. Beer, “Enhanced regulatory sequence prediction using gapped k-mer features,” *PLoS Comput. Biol.*, vol. 10, no. 7, Art. no. e1003711, 2014.
[7] B. Alipanahi, A. Delong, M. T. Weirauch, and B. J. Frey, “Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning,” *Nat. Biotechnol.*, vol. 33, no. 8, pp. 831–838, 2015.
[8] H. R. Hassanzadeh *et al.*, “MotifMark: Finding regulatory motifs in DNA sequences,” in *Proc. 39th Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.03321
[9] Rosalind, “Rosalind: A platform for learning bioinformatics through problem solving.” [Online]. Available: https://rosalind.info (accessed Jun. 21, 2026).
[10] P. Compeau and P. Pevzner, *Bioinformatics Algorithms: An Active Learning Approach*. La Jolla, CA, USA: Active Learning Publishers, 2014.
[11] R. W. Hamming, “Error detecting and error correcting codes,” *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, 1950.
[12] P. A. Pevzner and S.-H. Sze, “Combinatorial approaches to finding subtle signals in DNA sequences,” in *Proc. 8th Int. Conf. Intell. Syst. Mol. Biol. (ISMB)*, 2000, pp. 269–278.
[13] T. L. Bailey and C. Elkan, “Fitting a mixture model by expectation maximization to discover motifs in biopolymers,” in *Proc. 2nd Int. Conf. Intell. Syst. Mol. Biol. (ISMB)*, 1994, pp. 28–36.
[14] J. Zhou and O. G. Troyanskaya, “Predicting effects of noncoding variants with deep learning–based sequence model,” *Nat. Methods*, vol. 12, no. 10, pp. 931–934, 2015.
[15] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[16] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.